

LETTER

Formal and Empirical Analysis of an Authorization Decision Inconsistency in SAGA

TODO AUTHOR[†], ??

SUMMARY We analyze an order-dependent authorization inconsistency in SAGA: a later wildcard allow can replace a more-specific deny because the matcher fails to retain the best pattern. In controlled executions, the implementation decision propagates through production consumer paths to token issuance and storage. Under the fixed divergence, ProVerif reports ChatAccept reachable and the prior-TokenIssue correspondence true.

key words: authorization, access control, formal verification, AI agents, ProVerif

1. Introduction

SAGA [1] uses contact policies and access-control tokens to authorize communication between registered agents. Authentication establishes identity but does not imply permission under the receiver's contact policy.

We identify an order-dependent inconsistency in the production matcher. Under the documented most-specific semantics, the specific deny should be selected over the wildcard allow. Instead, the matcher returns the later wildcard allowance; reversing the rule order returns denial.

In controlled executions, the implementation decision propagates through production consumer paths to token issuance and storage. A separate symbolic model analyzes acceptance under the supplied divergence.

Our contributions are (1) identifying and explaining the matcher inconsistency, and (2) characterizing its consequences through controlled production-consumer propagation and conditional symbolic analysis.

(1) Related work.

Prior work studies policy-language semantics [2] and policy verification/change impact [3]; SAGA itself analyzes protocol properties [1] using ProVerif [4]. Our analysis instead concerns a concrete implementation deviation from the specified rule-selection semantics and downstream consumer effects, complementing SAGA's protocol-level guarantees under its model assumptions.

2. Authorization Decision Inconsistency

(1) Workflow and policy semantics.

An initiator requests contact through SAGA's Provider. The receiver's ordered policy maps identity patterns to integer budgets. Provider and receiver independently call the production matcher on the initiator. The Provider gates one-time

Table 1 Specification and production-matcher results under two rule orders.

Rule order	Spec. budget	Matcher budget
Specific deny, then wildcard allow	-1	10
Wildcard allow, then specific deny	-1	-1

key (OTK) release; the receiver rechecks its policy before token issuance.

For a fixed policy and valid identity, SAGA v2, Section IV-D.1 [1], specifies that the most-specific matching rule is selected; the implementation scores patterns with `aid_specificity()`. A positive budget permits the request; a negative budget denies it. We do not consider equal-specificity conflicts, policy changes during evaluation, or zero budgets.

(2) Root cause and observation.

The tested production matcher is `match()` in `saga/common/contact_policy.py`, unchanged from upstream SAGA revision 7372111, also the matcher-test baseline in our study copy.[†] Its `best_pattern` starts at `None`, whose specificity is `-1`. A matching rule with greater specificity updates the budget but never `best_pattern`. Thus later matching rules still compare against `-1`, allowing even a less-specific wildcard to overwrite the budget. In the recorded case, `aid_specificity()` scores `alice@example.com:*` as 70 and `*` as 0; their budgets are `-1` and 10, respectively.

An independent finite specification evaluator selects the specific denial for both orders in Table 1. When specific deny precedes wildcard allow, the specification denies with budget `-1`, but the production matcher returns 10 and permits. The reverse order returns the specified denial, establishing order dependence. These calls establish only a local decision inconsistency, not a deployed exploit, message acceptance, or whole-system failure.

3. Formal Consequence Analysis

(1) Decision input.

The symbolic model is instantiated with the recorded divergence: the specification denies the request, while the

[†]TODO: affiliation and postal address.
DOI: 10.1587/transfun.E0.A.1

[†]Upstream: github.com/gsiros/saga; study copy and artifacts: github.com/g1020268262-a11y/saga.

observed implementation permits it. ProVerif takes these fixed facts as assumptions; it does not execute Python or the production matcher, discover the inconsistency, or establish implementation refinement.

(2) Model assumptions.

The initiator has a valid registered identity, follows the prescribed symbolic peer process, and cannot alter the receiver's fixed policy; reachability is not an arbitrary-adversary reachability claim. We trust the Provider and Certificate Authority (CA), assume cryptographic checks are effective, and assume private keys remain uncompromised. A private symbolic channel abstracts authenticated, confidential transport.

In the symbolic receiver, $\text{TokenIssue}(R, I, t)$ occurs after inserting receiver R , initiator I , encrypted token t , and I 's public access-control key into ActiveToken , before sending t . For a private-channel message m , $\text{ChatAccept}(R, I, t, m)$ occurs only after checking the fixed initiator identity and an ActiveToken entry matching receiver, initiator, presented token, and sender public key.

(3) Queries and results.

The study repository's archived ProVerif 2.05 run (proofs/evidence, 22 September 2026) reports ChatAccept reachable under the fixed divergence. Separately, ProVerif reports the following non-injective correspondence as true:

$$\text{ChatAccept}(R, I, t, m) \implies \text{TokenIssue}(R, I, t)$$

It requires prior TokenIssue for the same receiver R , initiator I , and encrypted token t used by ChatAccept . In this fixed-divergence model, acceptance and the issuance correspondence coexist with specification denial: token-origin consistency alone does not establish policy compliance. The witness includes issuance; no standalone TokenIssue reachability query exists.

The empirical and symbolic executions are compared only at the level of authorization decisions and token lifecycle events; no object-level refinement is established. Only the symbolic model reaches message acceptance.

4. Consumer-Side Propagation Validation

(1) Environment.

At consumer-run revision 8cc5dd7, the relevant matcher, Provider, receiver, and token-cryptography files are identical to upstream revision 7372111. The harness invokes the production Provider /access route and receiver handler to exercise matcher evaluation, OTK release and consumption, token generation and sending, and `active_tokens` storage. Fixtures provide signed registration material, local certificates, and in-memory transport; production cryptographic checks remain active. Wrappers record and return original matcher results; fixtures preset neither authorization decisions nor resulting token state. After token sending and storage, a boundary stub intercepts the call before the original `receive_conversation` runs.

Table 2 Divergent-case propagation through production consumer paths; message acceptance was not executed.

Evidence point	Divergent case result
Specification	Deny (−1)
Provider matcher	Allow (10)
Provider /access	HTTP 200; OTK released
Receiver matcher	Allow (10)
Receiver	OTK consumed
Endpoint	Token issued and stored

(2) Cases and controls.

Provider and receiver use identical policies. The divergent case places specific deny (−1) before wildcard allow (10): the specification denies, while the production implementation permits. The controls use specific then wildcard allowances (10 each), or wildcard allow (10) followed by specific deny (−1). Each case ran once with fresh state and one OTK. A/A and D/D denote allow/allow and deny/deny for specification/observed implementation, not Provider/receiver outcomes. A/A reaches token sending and storage; D/D returns HTTP 403, releases no OTK, leaves state unchanged, and never reaches the receiver, so it does not test receiver rejection.

(3) Result and evidence.

Table 2 shows both matchers returning 10, followed by OTK release and consumption, then generation, transmission, and storage of one token in `active_tokens`.

The empirical endpoint is token issuance and storage, not message acceptance: no application message is sent and the original `receive_conversation` is not invoked.

5. Scope and Conclusion

Our evidence is limited to one recorded SAGA case and the tested policies; it does not establish correctness for arbitrary policies or any Python–ProVerif refinement. Empirical execution stops before message acceptance; no deployed exploit is established. The inconsistency propagates through controlled production paths to token issuance and storage; symbolic ChatAccept is separately reachable under the fixed divergence.

References

- [1] G. Syros, A. Suri, J. Ginesin, C. Nita-Rotaru, and A. Oprea, “SAGA: A security architecture for governing AI agentic systems.” arXiv preprint arXiv:2504.21034v2, 2025. Version 2, revised 29 August 2025.
- [2] M.Y. Becker, C. Fournet, and A.D. Gordon, “Design and semantics of a decentralized authorization language,” 20th IEEE Computer Security Foundations Symposium (CSF), pp.3–15, IEEE, 2007.
- [3] K. Fisler, S. Krishnamurthi, L.A. Meyerovich, and M.C. Tschantz, “Verification and change-impact analysis of access-control policies,” 27th International Conference on Software Engineering (ICSE), pp.196–205, 2005.
- [4] B. Blanchet, “Modeling and verifying security protocols with the applied pi calculus and ProVerif,” Foundations and Trends in Privacy and Security, vol.1, no.1–2, pp.1–135, 2016.